InsightSQL — Technical Guide

> **Purpose:** This document explains the complete architecture, data flow, and design decisions behind InsightSQL — an agentic NL2SQL system for Northwind analytics. It is written for developers who need to understand, debug, or extend the system.

Table of Contents

1. What InsightSQL Does
2. High-Level Architecture
3. The Agent Pipeline
4. State Management
5. Disambiguation Agent
6. Domain Guard Agent
7. Retrieval Agent & RAG System
8. SQL Generation Agent
9. Validation Agent (4 Layers)
10. Execution Agent
11. Explanation Agent
12. Controller & Routing Logic
13. Frontend & User Interaction
14. Evaluation Framework
15. Key Design Decisions

---

1. What InsightSQL Does

InsightSQL converts a plain-English business question into a SQL query, executes it, and returns a human-readable answer.

Example:

```
User: "What are the top 5 products by revenue in 2024?"
      ↓
System: "Here are the top 5 products by total revenue..."
      ↓
SQL: SELECT p.product_name, SUM(od.unit_price * od.quantity * (1 -
od.discount)) AS revenue ...
```

Unlike a simple text-to-SQL model, InsightSQL uses a **multi-agent pipeline** that:

- Asks clarifying questions when the user is vague
- Guards against off-topic queries
- Retrieves relevant schema context using hybrid search
- Validates SQL through 4 independent layers before execution

- Falls back to a demo backend if PostgreSQL is unavailable
- Explains results in business-friendly language

---

2. High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                        USER INTERFACE (Streamlit)                    │
│                                                                      │
│   ┌──────────────┐   ┌──────────────────────┐   ┌──────────────────┐ │
│   │ Query Input  │   │ Clarification Form   │   │ Results / SQL / Trace │
│   └──────────────┘   └──────────────────────┘   └──────────────────┘ │
└──────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────┐
│                      CONTROLLER (LangGraph StateGraph)               │
│                                                                      │
│                                                                      │
│                                                                      │
│   START → Disambiguation → Domain Guard → Retrieval → SQL Generation │
│                                       ↓                    ↓         │
│                                 (uses RAG)          Validation ←─────┤
│                                       ↓                    ↓         │
│                                 Execution ←──────────── (retry loop) │
│                                       ↓                              │
│                                 Explanation → END                    │
│                                                                      │
│                                                                      │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
                                   │
                  ┌────────────────┼────────────────┐
                  ▼                ▼                ▼
┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│   Groq API   │   │  PostgreSQL  │   │   ChromaDB   │
│ (LLM Calls)  │   │ (Primary DB) │   │ (Vector Store) │
└──────────────┘   └──────────────┘   └──────────────┘
```

Technology Stack

Layer	Technology	Purpose
UI	Streamlit	Single-page web dashboard
Orchestration	LangGraph (`StateGraph`)	Agent pipeline with conditional routing
LLM	Groq API (Llama 3.3 70B)	SQL generation, explanation, disambiguation
Vector DB	ChromaDB (persistent)	Schema chunks + example storage

Layer	Technology	Purpose
Embeddings	sentence-transformers (`all-MiniLM-L6-v2`)	Semantic search
Database	PostgreSQL 16 (port 5433)	Primary data store (Northwind)
Fallback	pandas (hardcoded DataFrames)	Demo mode when Postgres is unavailable
Validation	sqlglot	SQL AST parsing and safety checks
Observability	LangSmith	Trace every agent node

---

3. The Agent Pipeline

InsightSQL breaks the problem into **7 specialized agents**. Each agent is a Python module with a single responsibility. The controller chains them together using a LangGraph `StateGraph`.

The 7 Agents

#	Agent	File	Uses LLM?	What It Does
1	**Disambiguation**	`agents/disambiguation_agent.py`	✅ Groq	Detects vague queries and asks clarifying questions
2	**Domain Guard**	`agents/domain_guard_agent.py`	❌ No	Rejects off-topic questions (e.g., "Who won the World Cup?")
3	**Retrieval**	`agents/retrieval_agent.py`	❌ No	Fetches relevant schema columns and example queries
4	**SQL Generation**	`agents/sql_generation_agent.py`	✅ Groq	Writes the actual `SELECT` statement
5	**Validation**	`agents/validation_agent.py`	❌ No	Runs 4 validation layers before execution
6	**Execution**	`agents/execution_agent.py`	❌ No	Runs SQL against Postgres or demo backend
7	**Explanation**	`agents/explanation_agent.py`	✅ Groq	Summarizes results in plain English

Why 7 Agents Instead of One?

A single monolithic LLM call would be simpler but has major drawbacks:

- **No retry logic** — if SQL is wrong, you start over
- **No guardrails** — it might execute dangerous queries
- **No transparency** — you cannot tell which step failed
- **No retrieval** — the LLM hallucinates table/column names

By splitting into agents, we get:

- **Modular retries** — Validation can send SQL back to Generation up to 3 times
- **Safety** — Domain Guard blocks off-topic queries before any SQL is written
- **Observability** — LangSmith traces each node independently
- **Cost control** — Only 3 of 7 agents use an LLM; the rest use deterministic code

---

4. State Management

All agents communicate through a single shared state object: `AgentState`.

File

`backend/app/schemas/state.py`

What Is AgentState?

`AgentState` is a `TypedDict` with 30+ fields. Think of it as a "passport" that travels with your request through every agent. Each agent reads what it needs and writes what it produces.

Key Fields

```
class AgentState(TypedDict):
    # --- Input / Output ---
    query: str                    # Original user question
    refined_query: str            # Query after disambiguation
    sql: str                      # Generated SQL
    result: list[dict]            # Query results (rows)
    explanation: str              # Human-readable answer
    error: str                    # Error message (if any)

    # --- Disambiguation ---
    is_ambiguous: bool
    clarification_question: str   # What to ask the user
    clarification_attempts: int   # How many rounds so far (max 2)
    pending_clarification: bool   # Waiting for user input?
    user_clarification: str       # User's answer
    applied_clarification: str    # Merged context

    # --- RAG ---
    retrieved_schema_chunks: list[str]  # Top-3 relevant columns
    retrieved_examples: list[str]       # Top-2 example Q→SQL pairs
    use_rag: bool                       # Toggle RAG on/off

    # --- Validation ---
```

```
    validation_layers_triggered: list[str]
    failed_layer: str
    use_validation_layers: bool   # Toggle validation on/off

    # --- Execution ---
    data_source: str              # "postgres" or "demo"
    cardinality_warning: bool     # Too many rows returned?
    last_sql: str                 # Previous attempt (for retry prompts)
```

Why TypedDict?

LangGraph uses `TypedDict` for state because it provides:

- **Type checking** — Editors catch missing fields
- **Runtime validation** — LangGraph validates state transitions
- **Clear contracts** — Every agent documents what it expects and produces

---

5. Disambiguation Agent

File

`backend/app/agents/disambiguation_agent.py`

Purpose

Detect when a user asks a vague or underspecified question, and either:

1. **Ask for clarification** (up to 2 times), or
2. **Apply a default assumption** and proceed

Example Flow

```
User: "Show me sales by region"
      ↓
Disambiguation Agent:
  - "sales" could mean revenue, quantity, or order count
  - "region" could mean employee region or customer region
  → is_ambiguous = true
  → clarification_question = "Do you mean revenue or order count?"
      ↓
UI shows clarification form
      ↓
User: "revenue"
      ↓
applied_clarification = "Show me revenue by region"
```

Implementation

1. **LLM Prompt** — The agent sends the query + a schema summary to Groq. The LLM returns a JSON object:

   ```
   {
     "is_ambiguous": true,
     "clarification_question": "...",
     "default_assumption": "..."
   }
   ```

2. **Context-Aware Merge** — When the user replies, `merge_clarification()` intelligently merges the clarification into the original query rather than replacing it blindly.

3. **Deterministic Fallback** — If the Groq API fails, the agent falls back to keyword heuristics (e.g., checking for missing time ranges, aggregations, or table references).

4. **Attempt Cap** — After 2 clarification rounds, the system stops asking and uses the `default_assumption` to proceed.

Prompt Location

`backend/app/prompts/disambiguation.py`

---

6. Domain Guard Agent

File

`backend/app/agents/domain_guard_agent.py`

Purpose

Prevent the system from wasting LLM tokens and user patience on completely off-topic queries.

How It Works

This is a **deterministic** agent — no LLM call. It uses simple keyword matching:

```
# Pseudocode
if query_contains_any(NORTHWIND_KEYWORDS):
    in_scope = True
else:
    in_scope = False
    explanation = "I can only answer questions about Northwind business
data..."
```

The keyword list covers Northwind concepts: orders, customers, products, employees, revenue, shipping, suppliers, categories, territories, etc.

Why Not Use an LLM Here?

Keyword filtering is:

- **Instant** — no API latency
- **Free** — no token cost
- **Deterministic** — same query always yields same result
- **Good enough** — most off-topic queries are obviously unrelated

For borderline cases, the system errs on the side of allowing the query through. The SQL Generation agent will simply fail to write valid SQL, and the Validation agent will catch it.

---

7. Retrieval Agent & RAG System

Files

- `backend/app/agents/retrieval_agent.py`
- `backend/app/rag/retrieval.py`
- `backend/app/rag/index.py`
- `backend/app/rag/chunks.py`
- `backend/app/rag/examples.py`

Purpose

Give the SQL Generation agent the **minimal but sufficient** schema context it needs to write correct SQL. Without this, the LLM hallucinates column names and table relationships.

The Northwind Schema

The database has **15 tables** and **13 foreign keys**. Showing the LLM all 15 tables would:

- Exceed context limits
- Confuse the model with irrelevant columns
- Increase cost and latency

So we retrieve only the most relevant columns and relationships.

Hybrid Retrieval Strategy

```
User Query: "top products by revenue in 2024"
            │
     ┌──────┴──────┐
     ▼             ▼
Semantic       BM25
(ChromaDB)    (rank_bm25)
     │             │
     └──────┬──────┘
            ▼
    Hybrid Merge
    (semantic priority)
```

```
            |
            ▼
Top 3 schema chunks  +  Top 2 examples
```

1. Semantic Search (ChromaDB)

- **Collection:** `schema_chunks`
- **Contents:** One chunk per column, enriched with FK info

  ```
  "Table: orders | Column: order_date | Type: date | Description: When the order was placed"
  "Table: order_details | Column: unit_price | Type: numeric | FK: products.product_id"
  ```

- **Embedding:** `all-MiniLM-L6-v2` (384-dimensional, fast, good for schema matching)
- **K:** 3 chunks

2. BM25 Lexical Search

- Uses `rank_bm25` over the same chunk corpus
- Catches exact keyword matches that semantic search might miss (e.g., "order_id", "customer_id")

3. Hybrid Merge

- Combines both result sets
- **Semantic results take priority** — they are ranked higher in the merged list
- Final output: up to 3 schema chunks + 2 example queries

Example Retrieval

Query: "Which employee has the highest total revenue?"

Retrieved Schema Chunks:

```
1. employees.employee_id, employees.first_name, employees.last_name
2. order_details.unit_price, order_details.quantity, order_details.discount
3. orders.employee_id → employees.employee_id (FK)
```

Retrieved Examples:

```
Q: "Top 5 customers by total revenue"
SQL: SELECT c.customer_id, SUM(...) FROM customers c JOIN orders o ...
```

Example Store

`backend/app/rag/examples.py` contains **18 hand-written Q→SQL pairs** covering:

- Simple SELECT
- Two-table JOIN
- Multi-table JOIN (3+)
- Aggregation (SUM, COUNT, AVG)
- GROUP BY + HAVING
- Time-based queries

Index Building

Run `scripts/build_rag_index.py` to rebuild the ChromaDB index from scratch. This reads:

- `backend/app/rag/chunks.py` (schema chunks)
- `backend/app/rag/examples.py` (example queries)

And writes to `.rag_index/` (persistent ChromaDB directory).

---

8. SQL Generation Agent

File

`backend/app/agents/sql_generation_agent.py`

Purpose

Write a syntactically correct PostgreSQL `SELECT` query.

Input

The agent receives:

1. `refined_query` — the disambiguated user question
2. `retrieved_schema_chunks` — top-3 relevant columns/tables
3. `retrieved_examples` — top-2 similar Q→SQL pairs
4. `last_sql` — previous attempt (only during retries)
5. `last_error` — validation error from previous attempt (only during retries)

The Prompt

`backend/app/prompts/sql_generation.py` assembles a system prompt that includes:

- **Schema context** — the retrieved chunks
- **Foreign key rules** — explicit join directions (e.g., "orders.customer_id joins to customers.customer_id")
- **Time handling rules** — use `EXTRACT(YEAR FROM ...)` for year filtering
- **Example conflict guard** — "Do not copy the example SQL directly; adapt it to the user's question"
- **Retry format** — if this is a retry, the prompt shows `PREVIOUS SQL` and `ERROR` sections

Retry Loop

If Validation rejects the SQL, the controller routes back to SQL Generation (max 3 retries). Each retry includes the previous SQL and the error message, allowing the LLM to self-correct.

Example Retry Prompt

```
You are a PostgreSQL expert. Write a SELECT query for:
"Top 5 products by revenue in 2024"

Schema context:
[retrieved chunks here]

PREVIOUS SQL:
SELECT product_name, SUM(unit_price * quantity) ...

ERROR:
Semantic validation failed: Missing GROUP BY clause for non-aggregated
column "product_name"

Write a corrected query:
```

---

9. Validation Agent (4 Layers)

File

`backend/app/agents/validation_agent.py`

Purpose

Catch bad SQL **before** it touches the database. This is the safety net of the system.

Why 4 Layers?

Each layer catches different categories of errors. They run in sequence, and if any layer fails, the SQL is rejected.

```
SQL from Generation Agent
        │
        ▼
┌──────────────────────────────────────────┐
│ Layer 1: SAFETY (sqlglot AST)            │
│   - Is it a SELECT statement?            │
│   - No DELETE, DROP, INSERT, UPDATE?     │
│   - No dangerous functions?              │
└──────────────────────────────────────────┘
        │ Pass
        ▼
┌──────────────────────────────────────────┐
│ Layer 2: SCHEMA (AST + metadata)         │
│   - Do all tables exist?                 │
```

```
│   - Do all columns exist?                │
│   - Are aliases resolved correctly?      │
└──────────────────────────────────────────┘
        │ Pass
        ▼
┌──────────────────────────────────────────┐
│ Layer 3: SEMANTIC                        │
│   - Is GROUP BY correct?                 │
│   - Are aggregates used properly?        │
│   - No missing HAVING vs WHERE?          │
└──────────────────────────────────────────┘
        │ Pass
        ▼
┌──────────────────────────────────────────┐
│ Layer 4: EXPLAIN                         │
│   - Does PostgreSQL accept the plan?     │
│   - No type mismatches?                  │
│   - No missing FK constraints?           │
└──────────────────────────────────────────┘
        │ Pass
        ▼
  → Execution Agent
```

Layer 1: Safety (sqlglot)

Uses sqlglot to parse the SQL into an Abstract Syntax Tree (AST).

Checks:

- Root node must be `SELECT`
- No `DELETE`, `DROP`, `INSERT`, `UPDATE`, `ALTER`, `CREATE`, `TRUNCATE`
- No dangerous functions (`pg_read_file`, `pg_exec`, etc.)

Why this matters: Even if the LLM is well-behaved, prompt injection or model drift could produce destructive SQL.

Layer 2: Schema (AST + Metadata)

Cross-references the AST against `backend/app/db/northwind_full_schema.py`.

Checks:

- Every table in `FROM` / `JOIN` exists in `TABLE_COLUMNS`
- Every column exists in its referenced table
- Table aliases are resolvable (e.g., `o.order_id` maps to `orders`)

Layer 3: Semantic

Catches logical SQL errors that are syntactically valid but semantically wrong.

Checks:

- If `SELECT` contains aggregate functions (`SUM`, `COUNT`, etc.), non-aggregated columns must appear in `GROUP BY`
- `HAVING` is not used where `WHERE` should be (and vice versa)

Layer 4: EXPLAIN

Sends `EXPLAIN (FORMAT JSON) <sql>` to PostgreSQL (or the demo backend).

Checks:

- PostgreSQL can build a query plan without errors
- Catches type mismatches (e.g., comparing `date` to `text`)
- Catches missing implicit joins or constraint violations

Failure Handling

If any layer fails:

```
state["is_valid"] = False
state["failed_layer"] = "semantic"  # or "safety", "schema", "explain"
state["error"] = "Semantic validation failed: Missing GROUP BY ..."
```

The controller then routes back to SQL Generation for a retry.

---

10. Execution Agent

File

`backend/app/agents/execution_agent.py`

Purpose

Run the validated SQL and return results.

Dual Execution Backend

```
Is PostgreSQL healthy?
        │
   Yes ─┴─ No
   │         │
   ▼         ▼
Postgres   Demo Executor
(psycopg2)  (pandas)
   │         │
   └─────────┘
        │
        ▼
   Rows + Cardinality Check
```

Path A: PostgreSQL

- Uses `psycopg2` via `backend/app/db/connection.py`
- Runs the SQL with a statement timeout (configurable via `STATEMENT_TIMEOUT_MS`)
- Returns rows as a list of dictionaries

Path B: Demo Executor

If PostgreSQL is unreachable, the system falls back to a **pattern-matching SQL executor** that runs on hardcoded pandas DataFrames.

File: `backend/app/db/demo_executor.py`

It supports 8 common query patterns:

1. Simple `SELECT * FROM table`
2. `SELECT col1, col2 FROM table`
3. `SELECT ... WHERE ...`
4. `SELECT ... ORDER BY ... LIMIT ...`
5. `SELECT ... JOIN ...`
6. `SELECT ... GROUP BY ...`
7. `SELECT ... Aggregate functions`
8. Time-based filtering (`WHERE EXTRACT(YEAR FROM ...)`)

Why have a demo backend?

- **Zero setup** — new users can run the app without Docker/Postgres
- **Resilience** — the app works even during database outages
- **Fast evaluation** — no DB connection needed for unit tests

Cardinality Warning

If a query returns more than a threshold number of rows (default: 100), the agent sets:

```
state["cardinality_warning"] = True
```

This warns the UI that the result set is large.

---

11. Explanation Agent

File

`backend/app/agents/explanation_agent.py`

Purpose

Convert raw SQL rows into a business-friendly paragraph.

Input

- Original `query`
- Generated `sql`
- `rows` — the query results

Output

```
"The top 5 products by revenue in 2024 are:
1. Côte de Blaye — $120,450
2. Thüringer Rostbratwurst — $98,320
3. ..."
```

Why Use an LLM Here?

Formatting tabular data into natural language is highly contextual. The LLM decides:

- Whether to show a bullet list, a paragraph, or a comparison
- How to round numbers and format currencies
- Whether to highlight trends or outliers

---

12. Controller & Routing Logic

File

`backend/app/controller.py`

Purpose

Orchestrate the 7 agents into a cohesive pipeline with conditional branching.

The Graph

```python
from langgraph.graph import StateGraph

builder = StateGraph(AgentState)

# Add nodes
builder.add_node("disambiguation", disambiguation_agent)
builder.add_node("domain_guard", domain_guard_agent)
builder.add_node("retrieval", retrieval_agent)
builder.add_node("sql_generation", sql_generation_agent)
builder.add_node("validation", validation_agent)
builder.add_node("execution", execution_agent)
builder.add_node("explanation", explanation_agent)

# Set entry point
builder.set_entry_point("disambiguation")
```

```
# Add conditional edges
builder.add_conditional_edges("disambiguation",
_route_after_disambiguation)
builder.add_conditional_edges("domain_guard", _route_after_domain_guard)
builder.add_conditional_edges("validation", _route_after_validation)

# Compile
workflow = builder.compile()
```

Routing Rules

From Node	Condition	Route To
Disambiguation	`pending_clarification = true`	**END** (await user)
Disambiguation	clear OR max attempts reached	**Domain Guard**
Domain Guard	`out_of_scope = true`	**Explanation** (reject message)
Domain Guard	in scope	**Retrieval**
Validation	`is_valid = true`	**Execution**
Validation	`is_valid = false` + retryable + retries < 3	**SQL Generation** (retry)
Validation	`is_valid = false` + exhausted	**Explanation** (error message)

Entry Point

```
def run_agent_pipeline(
    query: str,
    max_attempts: int = 3,
    use_rag: bool = True,
    use_validation_layers: bool = True,
    prior_state: AgentState | None = None,
    user_clarification: str | None = None
) -> AgentState:
    """Main entry point for the entire pipeline."""
```

This function is called by:

- The Streamlit frontend on every user query
- The evaluation framework for batch testing

Caching

The graph structure is built once and cached via `@lru_cache` to avoid reconstructing the workflow on every request.

---

13. Frontend & User Interaction

File

`frontend/streamlit_app.py`

Layout

```
┌──────────────────────────────────────────────────────────────┐
│  InsightSQL — Agentic NL2SQL for Northwind Analytics         │
│                                                              │
│  [Business question input           ] [Run Query]            │
│                                                              │
│   ┌──────────────────────────────────────────────────────┐   │
│   │ Clarification (shown only when pending)              │   │
│   │ "Do you mean revenue or order count?"                │   │
│   │ [Your answer                        ] [Submit]       │   │
│   └──────────────────────────────────────────────────────┘   │
│                                                              │
│  Explanation: The top 5 products by revenue...               │
│  ⚠️ Cardinality Warning: 1,247 rows returned                 │
│  Data Source: PostgreSQL                                     │
│                                                              │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐    │
│   │ Agent Trace  │   │   Results    │   │ Retrieved    │    │
│   │ (dataframe)  │   │ (dataframe)  │   │ Schema       │    │
│   └──────────────┘   └──────────────┘   └──────────────┘    │
│                                                              │
│  [+] Generated SQL         [+] Agent State (JSON)            │
│                                                              │
└──────────────────────────────────────────────────────────────┘

Sidebar:
  ┌──────────────────┐
  │ System Status    │
  │ ● PostgreSQL Online │
  │                  │
  │ Agent Stack      │
  │ - LangGraph      │
  │ - Groq (Llama 3.3) │
  │ - LangSmith      │
  │                  │
  │ Demo Queries     │
  │ [Top products]   │
  │ [Revenue by region] │
  │ [Best employees] │
  │ ...              │
  │                  │
  │ Query History    │
  │ 1. ...           │
  └──────────────────┘
```

Interaction Flow

1. **User enters query** (or clicks a demo button)
2. `run_agent_pipeline(query)` is called
3. If `pending_clarification` is returned, the UI shows the clarification form
4. **User submits clarification** → `run_agent_pipeline(prior_state, user_clarification)` is called
5. The UI renders all state fields: explanation, results, trace, SQL, etc.

Session State

Streamlit session state holds:

- `last_state` — most recent `AgentState` for rendering
- `pending_state` — paused state awaiting clarification
- `history` — `deque(maxlen=10)` of past queries

---

14. Evaluation Framework

Files

- `evaluation/golden_dataset.py`
- `evaluation/configurations.py`
- `evaluation/runner.py`
- `evaluation/metrics.py`
- `evaluation/normalize.py`
- `evaluation/report.py`
- `scripts/run_eval.py`

Purpose

Measure whether InsightSQL actually works, and compare the impact of different features (RAG, validation layers).

Golden Dataset

50 hand-verified questions across 6 categories:

Category	Count	Example
Simple SELECT	10	"List all customers"
Two-table JOIN	10	"Show orders with customer names"
Multi-table JOIN	10	"Products with supplier and category"
Aggregation	10	"Total revenue by country"
GROUP BY + HAVING	5	"Countries with more than 10 orders"
Time-based	5	"Orders placed in 2024"

Each question has:

- `gold_sql` — hand-written correct SQL
- `time_sensitive` flag — for special date handling

Configurations

The evaluator runs each question under 3 configurations:

1. **Baseline** — `use_rag=False`, `use_validation_layers=False`
   - Raw LLM with no help
2. **RAG** — `use_rag=True`, `use_validation_layers=False`
   - LLM gets retrieved schema + examples
3. **Full** — `use_rag=True`, `use_validation_layers=True`
   - Complete pipeline

Metrics

Metric	Definition
Execution Accuracy	Result set matches golden result set (after normalization)
Execution Success	Query runs without error
Error Recovery	Failed query → retry → success
Latency (ms)	End-to-end time per query

Normalization

Before comparing result sets, `evaluation/normalize.py` applies:

1. **Sort** — order-independent comparison
2. **Round** — floating-point tolerance
3. **Frozenset** — deduplication of identical rows

This ensures that `ORDER BY` differences or minor rounding do not penalize correct answers.

Report

`evaluation/report.py` generates:

- `results.csv` — per-question, per-configuration results
- `summary.png` — matplotlib bar chart comparing accuracy across configurations

---

15. Key Design Decisions

1. Agentic vs. Monolithic

Decision: Split into 7 specialized agents orchestrated by LangGraph.

Why: Enables retries, guardrails, observability, and cost control. Each agent has a single responsibility.

Trade-off: More code and latency than a single LLM call, but far higher reliability.

---

2. Hybrid RAG (Semantic + BM25)

Decision: Combine ChromaDB semantic search with BM25 lexical search.

Why: Semantic search catches conceptual similarity ("sales" ≈ "revenue"), while BM25 catches exact ID matches ("order_id").

Trade-off: Slightly more complex than pure semantic search, but better recall for schema-heavy queries.

---

3. Four-Layer Validation

Decision: Safety → Schema → Semantic → EXPLAIN.

Why: Catches errors at increasing levels of sophistication. Catastrophic errors (injection) are caught immediately; subtle errors (missing GROUP BY) are caught before execution.

Trade-off: Adds ~50-200ms per query, but prevents bad SQL from hitting the database.

---

4. Dual Execution Backend

Decision: Support both PostgreSQL and a pandas demo backend.

Why: Zero-setup demos, resilience to DB outages, faster unit tests.

Trade-off: Demo executor only covers 8 query patterns; complex queries fail in demo mode.

---

5. Clarification Loop (Max 2)

Decision: Ask the user up to 2 clarifying questions before applying a default assumption.

Why: Balances specificity with user friction. Most vague queries need only 1 round of clarification.

Trade-off: Adds UI complexity (clarification form) and latency.

---

6. Ablation Flags (`use_rag`, `use_validation_layers`)

Decision: Make RAG and validation toggleable.

Why: Enables scientific evaluation. We can measure exactly how much each feature improves accuracy.

Trade-off: Slightly more branching logic in the controller.

---

7. LangSmith Tracing

Decision: Trace every agent node with LangSmith.

Why: Production debugging. When a query fails, we can inspect the exact state at every step.

Trade-off: Adds network overhead and requires `LANGSMITH_API_KEY`.

Appendix: File Structure Reference

```
backend/
├── app/
│   ├── agents/
│   │   ├── disambiguation_agent.py      # Clarification logic
│   │   ├── domain_guard_agent.py        # Scope checking
│   │   ├── retrieval_agent.py           # RAG orchestration
│   │   ├── sql_generation_agent.py      # LLM SQL writer
│   │   ├── validation_agent.py          # 4-layer validator
│   │   ├── execution_agent.py           # Query runner
│   │   └── explanation_agent.py         # Result summarizer
│   ├── controller.py                    # LangGraph orchestrator
│   ├── core/
│   │   └── config.py                    # Environment settings
│   ├── db/
│   │   ├── connection.py                # psycopg2 wrapper
│   │   ├── health.py                    # DB health checks
│   │   ├── demo_executor.py             # pandas fallback (8 patterns)
│   │   ├── demo_data.py                 # Hardcoded DataFrames
│   │   ├── northwind_full_schema.py     # Full schema metadata
│   │   └── northwind_schema.py          # Legacy keyword map
│   ├── prompts/
│   │   ├── disambiguation.py            # LLM prompt for clarification
│   │   └── sql_generation.py            # LLM prompt for SQL
│   ├── rag/
│   │   ├── index.py                     # ChromaDB persistent store
│   │   ├── chunks.py                    # Schema chunk builder
│   │   ├── examples.py                  # 18 Q→SQL examples
│   │   └── retrieval.py                 # Semantic + BM25 hybrid
│   ├── schemas/
│   │   └── state.py                     # AgentState TypedDict
│   └── services/
│       └── llm.py                       # Groq API wrapper
├── frontend/
│   └── streamlit_app.py                 # Web UI
evaluation/
├── golden_dataset.py                    # 50 test questions
├── configurations.py                    # Baseline / RAG / Full runners
├── runner.py                            # Batch evaluation loop
├── metrics.py                           # Accuracy / success / latency
├── normalize.py                         # Result set normalization
└── report.py                            # CSV + matplotlib output
scripts/
├── build_rag_index.py                   # Rebuild ChromaDB
└── run_eval.py                          # Run full evaluation
data/
```

```
└── postgres/
    └── init/
        └── 01_northwind_demo.sql        # Database init script
```

Document generated for InsightSQL. For questions or updates, refer to `docs/CODEMAPS/` for module-specific details.